

Computational Materials Chemistry Setup

A beginner guide for Linux and macOS

Teaching deck

Updated September 2026

Learning goals

- Identify the operating system and processor architecture.
- Install a system toolchain without mixing incompatible software stacks.
- Create an isolated Python environment for scientific work.
- Verify compilers, CMake, pkg-config, and OpenMPI.
- Choose a realistic Quantum ESPRESSO route for Linux or macOS.
- Run a small Python test and a minimal electronic-structure calculation.

Classroom rule

Run one step at a time and read its output. Never copy a command containing `sudo` until you understand what it changes.

The five layers

Layer	Purpose
System package manager	Compilers, build tools, MPI, and desktop apps
Conda environment	Python and packages isolated from the operating system
Scientific Python	Structures, analysis, plotting, notebooks, workflows
Quantum ESPRESSO	Electronic-structure and materials calculations
Visualization	Structures, trajectories, charge densities, outputs

Keep the layers clear: Conda is not the operating-system package manager.

Step 0: identify the machine

Linux

```
cat /etc/os-release  
uname -m
```

macOS

```
sw_vers  
uname -m
```

- x86_64: Intel/AMD 64-bit.
- arm64 on macOS: Apple Silicon (M-series).
- aarch64 on Linux: 64-bit ARM.

Why this matters

A package may exist for Intel macOS but not Apple Silicon. Check availability for the exact platform.

Use the package manager for your system

System	Manager	Typical command
Ubuntu / Debian	APT	<code>sudo apt install ...</code>
Fedora / RHEL family	DNF	<code>sudo dnf install ...</code>
Arch Linux	Pacman	<code>sudo pacman -S ...</code>
macOS	Homebrew	<code>brew install ...</code>

- Use only the instructions for your distribution.
- Package names differ; search the official package index when needed.
- Update package metadata before installing.

Ubuntu and Debian

```
sudo apt update
sudo apt install build-essential gfortran cmake pkg-config \
    openmpi-bin libopenmpi-dev git wget curl
```

Verify

```
gcc --version
gfortran --version
cmake --version
pkg-config --version
mpirun --version
```

What was installed?

C/C++ tools, a Fortran compiler, a build configurator, a library-query tool, and an MPI implementation.

Fedora and Arch Linux

Fedora

```
sudo dnf group install "Development Tools"
sudo dnf install gcc-gfortran cmake pkgconf-pkg-config \
    openmpi openmpi-devel git wget curl
```

Fedora may require loading its MPI environment; follow the package message or school administrator.

Arch Linux

```
sudo pacman -Syu
sudo pacman -S --needed base-devel gcc-fortran cmake \
    pkgconf openmpi git wget curl
```

If a package is not found, search the distribution's official package index instead of guessing.

macOS toolchain

1. Install current Apple Command Line Tools

```
xcode-select --install
```

2. Install Homebrew from <https://brew.sh>, then

```
brew update  
brew install gcc cmake pkg-config open-mpi git wget
```

3. Verify

```
brew --prefix  
brew info gcc open-mpi  
gfortran --version  
cmake --version  
mpirun --version
```

Apple Silicon note

Native Homebrew normally uses `/opt/homebrew`; Intel Homebrew commonly uses `/usr/local`.
Ask `brew --prefix` instead of hard-coding a path.

What each tool does

`gcc` Compiles C.

`gfortran` Compiles Fortran; Quantum ESPRESSO is mainly Fortran.

`cmake` Configures a build for the local machine and libraries.

`pkg-config` Reports compiler and linker flags for libraries.

`mpirun` Launches multiple MPI processes.

Important

Installing the toolchain does not install Quantum ESPRESSO. It makes a source build possible.

Install Conda and create an environment

Use Miniforge from <https://github.com/conda-forge/miniforge>. Match the installer to the architecture from Step 0.

```
conda --version
conda create -n materials -c conda-forge python=3.12 pip \
  numpy scipy matplotlib pandas jupyterlab ipykernel \
  ase pymatgen spglib seekpath phonopy h5py sympy \
  scikit-learn tqdm plotly rich requests
conda activate materials
which python
python --version
conda info --envs
```

Why an environment?

It keeps project packages separate from the operating system and from other projects.

Core scientific Python packages

Package	Use
NumPy, SciPy	Arrays, linear algebra, numerical methods
Matplotlib, Plotly	Static and interactive plots
Pandas	Tables and data cleaning
JupyterLab	Interactive notebooks
ASE	Atomic structures, calculators, workflows
pymatgen	Structures, symmetry, input/output analysis
spglib, seekpath	Symmetry and reciprocal-space paths
phonopy	Phonon workflows and analysis
h5py	HDF5 scientific data files
scikit-learn	Introductory machine learning

Verify and reproduce the environment

```
python -c "import numpy, scipy, matplotlib, pandas, ase, \
pymatgen, spglib, seekpath, phonopy, h5py; \
print('Materials Python stack OK')"\
jupyter lab
```

A successful import returns without a traceback. Stop JupyterLab with Ctrl+C.

Create an `environment.yml` for classmates:

```
name: materials
channels: [conda-forge]
dependencies:
  - python=3.12
  - numpy
  - scipy
  - matplotlib
  - jupyterlab
  - ase
  - pymatgen
  - spglib
  - phonopy
```

Recreate it with `conda env create -f environment.yml`.

Choose the QE route by platform

Platform	Beginner route	Reality check
Ubuntu / Debian Fedora / Arch	Repository, then Conda qe, then source Search official repository; Conda qe; source	Repository versions may be older Names and MPI integration vary
Linux ARM64 Intel macOS Apple Silicon	Check repository and Conda qe Check conda-forge osx-64 files Native source build or school Linux/HPC	Confirm linux-aarch64 Recheck availability before class Do not assume an osx-arm64 build exists

As of this deck's update, conda-forge lists qe for Linux x86-64, Linux AArch64, and Intel macOS, but not native Apple Silicon. Recheck the live package files.

Linux installation attempts

Ubuntu / Debian repository

```
sudo apt update  
apt show quantum-espresso  
sudo apt install quantum-espresso
```

Conda-forge, on listed platforms

```
conda search -c conda-forge qe  
conda install -n materials -c conda-forge qe
```

Use the exact package name

The conda-forge package is qe, not quantum-espresso. Inspect the current platform files before promising availability.

Apple Silicon macOS decision path

- ❶ Confirm `uname -m` prints `arm64`.
- ❷ Check the live conda-forge qe files for `osx-arm64`.
- ❸ If no native build exists, do not force an Intel package into a native environment.
- ❹ Use a native source build after developer tools are current, or use school Linux/HPC.

```
uname -m  
conda info  
conda search -c conda-forge qe
```

Rosetta is an advanced compatibility option, not the default classroom route. It adds an Intel stack and increases the risk of mixing MPI libraries.

Native source build: general outline

Download a stable release from <https://www.quantum-espresso.org/download-page/> and read that release's build instructions.

```
tar -xf qe-VERSION.tar.gz
cd qe-VERSION
mkdir build && cd build
cmake .. -DCMAKE_Fortran_COMPILER=gfortran
cmake --build . -j 4
ctest --output-on-failure
```

- Options depend on MPI and math libraries detected locally.
- Save the configuration summary and inspect warnings.
- A successful compilation is not enough; run the supplied tests.

Do not mix MPI stacks

```
which mpirun  
which mpifort  
which pw.x  
mpirun --version
```

- Conda QE should use compatible libraries from its solved environment.
- QE built with Homebrew OpenMPI should use the compatible Homebrew MPI.
- The first `mpirun` on PATH is not automatically correct.

Possible mismatch symptoms

Startup crashes, missing libraries, or messages mentioning MPI ABI, PMIx, ORTE, or incompatible libraries.

Verify Quantum ESPRESSO

```
which pw.x  
pw.x --version  
which ph.x  
which bands.x  
which projwfc.x
```

If MPI was installed and this build supports it:

```
mpirun -np 2 pw.x --version
```

On a laptop, begin with one or two processes. More processes can be slower and use more memory.

Visualization choices

Tool	Good for	Route
ASE GUI	Structures, quick inspection	Installed with ASE; <code>ase gui</code>
VESTA	Crystals and densities	Official VESTA download
OVITO	Trajectories, atomistic data	Official installer; macOS cask may exist
XCrySDen	QE structures and densities	Common on Linux; availability varies
Jmol	Molecules and teaching	Official Jmol distribution

On macOS, inspect before installing: `brew info --cask ovito`.

End-to-end test A: create a crystal

Save as test_materials.py:

```
from ase.build import bulk
from ase.io import write

si = bulk("Si", "diamond", a=5.43)
print(si)
print("Atoms:", len(si))
print("Cell volume (Å3):", round(si.get_volume(), 3))
write("silicon.cif", si)
print("Wrote silicon.cif")
```

Run it:

```
conda activate materials
python test_materials.py
ase gui silicon.cif
```

Success connects Python, ASE, file writing, and visualization.

End-to-end test B: minimal QE input

Obtain a trusted Si pseudopotential; record its source and license.

```
&CONTROL
  calculation='scf', prefix='si', pseudo_dir='./pseudo',
  outdir='./tmp'
/
&SYSTEM
 ibrav=2, celldm(1)=10.26, nat=2, ntyp=1, ecutwfc=30
/
&ELECTRONS
  conv_thr=1.0d-8
/
ATOMIC_SPECIES
  Si 28.0855 Si.UPF
ATOMIC_POSITIONS alat
  Si 0.00 0.00 0.00
  Si 0.25 0.25 0.25
K_POINTS automatic
  4 4 4 0 0 0
```

Run and inspect the QE test

```
mkdir -p pseudo tmp  
# Place the pseudopotential at pseudo/Si.UPF  
pw.x -in si.scf.in > si.scf.out  
tail -n 25 si.scf.out  
grep -E '!!JOB DONE' si.scf.out
```

- JOB DONE. indicates normal completion.
- The line beginning with ! reports the final total energy.
- The cutoff and k-point grid are teaching values, not convergence proof.

Scientific habit

Before publishing, test convergence with respect to energy cutoff, k-point sampling, cell size, and other relevant settings.

Common errors

Message or symptom	First response
command not found	Activate the environment; inspect PATH; use <code>which</code>
PackagesNotFoundInChannelsError	Check exact name and platform builds; do not force another architecture
Homebrew has no formula	Use <code>brew search</code> and the official formula site
Conda solve fails	Use a clean environment; avoid mixing channels; check architecture
Compiler cannot create executables	Update developer tools; inspect the first compiler/linker error
Missing shared library	Check that executable and libraries come from compatible stacks
MPI startup crash	Compare MPI path, compiler wrappers, and QE build source
No JOB DONE.	Find the first error above the final lines; validate input and paths

Diagnostic checklist

- ① Record OS and architecture.
- ② Record the active Conda environment.
- ③ Locate each executable.
- ④ Record versions.
- ⑤ Reduce to the smallest failing command.
- ⑥ Copy the complete first error, not only the last line.

```
uname -a
conda info --envs
which python gcc gfortran cmake mpirun pw.x
python --version
gfortran --version
mpirun --version
```

Final checklist

- ☐ OS and architecture recorded
- ☐ System package manager updated
- ☐ GCC/gfortran, CMake, pkg-config, and MPI verified
- ☐ `materials` environment created and activated
- ☐ Scientific Python imports pass
- ☐ Silicon CIF created and viewed
- ☐ QE route chosen for the actual platform
- ☐ `pw.x` and companion programs located
- ☐ QE test ends with `JOB DONE.`
- ☐ Versions and installation notes saved

Primary references

- Quantum ESPRESSO: <https://www.quantum-espresso.org/download-page/>
- QE user guide: https://www.quantum-espresso.org/Doc/user_guide/
- Conda environments:
<https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-environments.html>
- conda-forge qe builds: <https://anaconda.org/conda-forge/qe/files>
- Miniforge: <https://github.com/conda-forge/miniforge>
- Homebrew: <https://brew.sh> and <https://formulae.brew.sh>
- Linux packages: <https://packages.ubuntu.com>, <https://packages.fedoraproject.org>,
<https://archlinux.org/packages/>
- ASE: <https://ase-lib.org>

Package availability changes. Recheck these sources before each new class.

Ready to calculate

Start small, verify every layer, and keep a record of versions.